Lecture 8
Solutions

Homework 8: Clean up and ship

Solutions to Homework 8

```
$ bash codeHints/L08/setup_hw8.sh
$ cd practice
$ git clone hw8_server.git hw8
$ cd hw8
$ git log --oneline --graph --all --decorate
$ git switch feature
$ git rebase -i main
```

The editor opens with the five commits, oldest first (your IDs differ from here on):

```
pick 74e9302 Add greeting
pick b8385c7 wip
pick c59f243 fix typo
pick 3b4d390 add farewell
pick 83ae7a6 oops
```

Change the first word of lines 2 to 5, save and close:

```
pick 74e9302 Add greeting
fixup b8385c7 wip
fixup c59f243 fix typo
reword 3b4d390 add farewell
fixup 83ae7a6 oops
```

Git replays `Add greeting` first and stops with a conflict in `README.md` (you also see `HEAD (no branch)` in `git status`: normal during a rebase), because `main`'s newest commit also added a line there. In the file, the part above `=======` is `main`'s line and the part below is yours; keep both:

```
# Greeter
Maintained by the team.
Usage: run the greeter.
```

```
$ git add README.md
$ git rebase --continue
        # editor 1: message of "Add greeting": save and close
        # editor 2 (reword): change "add farewell" to "Add farewell"
$ git log --oneline --reverse main..feature
$ git push                        # rejected (non-fast-forward): expected
$ git push --force-with-lease
$ git log --oneline origin/hotfix -1
$ git switch main
$ git cherry-pick <that-id>
$ git push
$ printf 'shared-rebase=no\nsafer-force=force-with-lease\nnew-ids=rebase\n' \
    > ../hw8_answers.txt
```

8.2. `fixup` merges a commit into the one above and keeps the *upper* message, so `wip` and `fix typo` disappear into `Add greeting`, and `oops` into `Add farewell`. `reword` stops once more so that you can

fix the capital letter. If you used `squash` instead of `fixup`, Git opens an editor with combined messages, and you must delete the extra ones, otherwise the checker finds the wrong messages.

8.3. Two commits, `Add greeting` and `Add farewell`, on top of `Update README`. If `git log --oneline main..feature` shows five commits, you ran a plain rebase without `-i` or forgot to edit the list.

8.4. The rebase created copies of the commits, so the server's `feature` (with the old five) is no longer an ancestor of yours, and a normal push is rejected. `--force-with-lease` is allowed to replace it because the server's branch is exactly where you last saw it.

8.5. `git cherry-pick` copies the hotfix commit onto `main`: a *new* commit with the same change and message. The original stays on `hotfix`. `main` remains a straight line of four commits. (Merging `hotfix` would add the original commit and a merge commit, which the checker rejects.)

8.6. `shared-rebase=no` (golden rule), `safer-force=force-with-lease`, `new-ids=rebase` (merge keeps all existing commits and only adds one).

Common mistakes.

- Merging `main` into `feature` instead of rebasing: the history keeps five messy commits plus a merge commit.
- Leaving the rebase unfinished (after a conflict you must `git add` and `git rebase --continue`). `git status` says `rebase in progress`; the checker's last check fails until you finish or `git rebase --abort`.
- Using plain `git push --force`. It works here, but it is exactly the habit the lecture warns against; use `--force-with-lease`.
- Stuck in the vim editor: press `Esc`, type `:wq`, press Enter. (`:q!` quits *without saving*, which does not cancel a rebase: with an unchanged todo list every commit is simply replayed.) To cancel a rebase, run `git rebase --abort`, or delete all lines of the todo list and save.